Module 4, Lecture 1 — Work Breakdown, Dependencies, Estimation, Milestones, and Sprint Planning

> **Course:** Staff Engineer / Technical Lead Mentorship **Module:** 4 — Work breakdown structure, task backlog, dependencies, estimation, milestones, and sprint planning **Lecture:** 1 of Module 4 **Case study:** Planning the Spendwise MVP (React Native + FastAPI + PostgreSQL) **Format:** A one-way lecture. Everything is demonstrated for you, including the exercise, a review of a typical attempt, and a reference solution. Nothing here waits for your reply. **Builds on:** the PRD (Module 2), the Technical Design and API contracts (Module 3, Lecture 4), and the risk register and release gates (Module 5, Lecture 1).

Why this lecture, why now

You said "Continue." The course order puts **Module 4** before Module 5, and Module 5 Lecture 1 told you that Module 4 would make its effort guesses rigorous. So I'm teaching Module 4 now. The remaining Module 5 topics (security, performance, and quality gates in more depth) and Module 6 (deployment and operations) come after this.

Part 1 — Learning objective

After this lecture you should be able to:

1. **Build a work breakdown structure (WBS)** from a PRD, a technical design, and a risk register, including the work that people forget (infrastructure, tests, recovery drills, store submission).
2. **Create a backlog** whose items are ordered by value, risk, and dependency, starting with a **walking skeleton.**
3. **Map dependencies,** find the **critical path,** and separate *effort* from *waiting time.*
4. **Estimate** with ranges and stated assumptions, using decomposition, three-point (PERT) estimates, historical calibration, and spikes, and explain **why summed uncertainty is misleading.**
5. **Convert estimates into dates** through capacity, and reconcile an honest plan with a client's expectation.
6. **Define milestones** with objective exit criteria, tied to release gates.
7. **Plan sprints,** track progress, and **replan** using actual data.

Part 2 — Real-world scenario

The PRD is approved. The Technical Design is done. The API contract and the risk register exist. Meera sends a message:

> **Meera:** *"Great, so we're all set! You said two months. Can you confirm: launch in 8 weeks? My friends are already asking. Also, can you send me a plan?"*

A junior developer opens a spreadsheet, lists the features from the PRD (sign-in, add expense, summary, …), puts a number next to each, adds them up, and answers "Yes."

Look at what that answer silently assumes:

- That the *feature list is the whole job.* (What about the build pipeline, the environments, the restore drill, the store submission, the runbook?)
- That the numbers are *exact.* (Nobody said how confident they are.)
- That you can work *every hour of every week.* (What about client calls, context switching, illness, learning React Native?)
- That *nothing outside your control* will be late. (The app store accounts. Meera's privacy policy. A decision about how users sign in.)
- That *a date is a promise* rather than a forecast.

An experienced lead hears the same message and sees a different problem: **"I have been asked to confirm a date before I have a plan, and I don't yet know if the date and the scope can both be true."** The honest answer is not yes or no. It is: *"Give me a few days to break the work down and estimate it, then I'll tell you what I know, what I'm assuming, and what the options are."*

That is the lecture: how to produce a plan you can defend, including when the plan says "not in eight weeks."

Part 3 — The lead's thought process

3.1 What planning is for

First, a mindset correction. People think a plan is a prediction: "it will take 16 weeks." It is better to think of a plan as four tools:

Purpose	What it gives you
Shared expectation	Everyone (you, Meera) knows roughly what will be delivered and when, and what the date depends on
Early feasibility check	You discover *now* that scope and date conflict, while changing scope is cheap
Sequencing	You decide the order, doing risky and uncertain things *first*, while there is time to react
A baseline to track against	You can tell, week by week, whether you are ahead or behind, and by how much

A few terms to keep apart:

- **Estimate:** a judgment of how much *effort* work will take, ideally as a **range** with stated assumptions.
- **Forecast:** a prediction of *when* something will finish, based on estimates and capacity. It changes as you learn.
- **Commitment:** a promise. You make one only when you have enough information, and you should commit to a **confidence level** (for example "very likely by week 16"), not to a single optimistic point.
- **Target:** a date someone *wants*. A target is an input to planning, not an output.

The cone of uncertainty. Early in a project, estimates are inherently wide. Commonly cited figures say that at the initial concept stage the true effort can be several times larger or smaller than the estimate, and that the range narrows as requirements, design, and then real work progress. The exact numbers vary by study, but the shape is reliable: **the more you have learned, the narrower the honest range.** We have a PRD, a design, and contracts, so we are well into the cone, but we have *zero* build history, so the range is still wide. A lead says so.

Two well-documented human biases make plans optimistic: the **planning fallacy** (people underestimate the time for their own tasks, even when they know similar tasks overran before) and **anchoring** (the first number mentioned, such as the client's "8 weeks," pulls every later estimate toward it). A good process is built to defend against both.

DBA connection. You already know this from capacity planning and maintenance windows. You never say "the migration takes 10 minutes" from a hunch; you rehearse it on a copy, measure it, add a margin for the unexpected, and say "plan for 20, and I'll confirm after the rehearsal." Project planning is the same discipline.

3.2 The work breakdown structure (WBS)

A **WBS** is a hierarchical decomposition of the **deliverables** of the project into smaller and smaller pieces, until each piece is small enough to estimate, assign, and track.

Four rules make a WBS useful:

1. **Deliverable-oriented, not activity-oriented.** "Create expense endpoint with idempotency and contract tests" is a deliverable. "Coding phase" is not. Phases hide work; deliverables expose it.
2. **The 100% rule.** The WBS must include **all** the work in scope (100%) and **nothing outside** the scope. If work is not in the WBS, it will not be estimated, scheduled, or noticed, until it bites.
3. **Right-sized work packages.** A package should be small enough to estimate with some confidence and to finish in a short time. A practical rule: **roughly 4 to 20 hours.** Smaller packages cost more to manage; larger ones hide uncertainty. Anything bigger than about 3 days gets split or gets a spike first.
4. **Every package has a "done" definition** (acceptance criteria and evidence), so that "90% done" is not a thing.

Where the WBS comes from. A WBS invented from imagination misses things. A lead derives it from the documents you already have:

Source	What it contributes
PRD (functional requirements, user stories)	Feature work packages
Technical Design (components, flows, operations)	Foundations: repository, CI/CD, environments, observability, migrations
API contracts and ERD	Backend endpoints, schema, contract tests
NFRs	Performance, security, recovery, observability tasks
Risk register	**Mitigation work** (the restore drill, authorization tests, probes)
Release gates	**Evidence work** (the load-test report, the drill logs, the checklists)
Dependencies	Client tasks, external waits, store submission

Pay attention to the last four rows. **That is the work most estimates leave out.** Remember the lesson from Module 5: the release gates are *scope*. A plan that does not include the work to pass the gates is a plan that will fail them or skip them.

The forgotten categories. When I review a WBS, I check that each of these appears:

- Repository, tooling, linting, type checks, pre-commit hooks
- CI pipeline (tests with a real PostgreSQL, caching)
- Environments (staging and production), secrets, TLS, domain
- Observability: logs, error tracking, uptime monitor, alerts, **testing the alerts**
- Database: schema, migrations, seed data, backup configuration
- Security and privacy work: dependency scanning, secrets scan, log hygiene, account deletion, retention
- **Recovery drills:** restore, rollback
- Performance work: data seeding, load tests, tuning
- Mobile release plumbing: signing, build pipeline, test tracks, store listing, privacy disclosures
- Documentation: README, runbook, onboarding for the pilot
- Defect fixing and rework (time for the bugs you will find)
- **Learning time** for unfamiliar technology
- Project overhead: planning, demos, client communication (covered by the capacity factor, not as tasks, so as not to count it twice)

Vertical slices, not horizontal layers. Organise the *feature* work so that each slice cuts through every layer (app screen, API, database) and delivers something usable. Building "all the screens, then all the APIs, then all the database" delivers nothing testable until the end, and integration problems all appear at once, late.

A work package dictionary entry (one short paragraph per package) prevents arguments later:

> **3.1 Create expense (backend).** *Deliverable:* `POST /v1/expenses` per contract v0.1, idempotent by client UUID. *Includes:* validation, repository, transaction, error mapping, the 12 contract tests, OpenAPI snapshot. *Excludes:* the mobile screen (4.3). *Done when:* all 12 contract tests pass in CI against real PostgreSQL; deployed to staging; contract file updated. *Depends on:* 1.2, 1.5, 2.2.

3.3 The backlog, and starting with a walking skeleton

The **backlog** is the ordered list of work packages (items) waiting to be done. Each item should have an ID, a title, a type, a link to a requirement or risk, an estimate, its dependencies, and a

"done" definition.

Item types (so nothing is invisible):

Type	Example
Feature slice	Create expense, end to end
Enabler (technical work users never see)	CI pipeline; migrations
Spike	A time-boxed experiment to learn something: "1 day: can the React Native secure-storage library keep tokens across app reinstalls on both platforms?"
Risk mitigation	Restore drill; two-user authorization tests
Defect	Bugs found during the work
Client task	Create store accounts; supply privacy-policy text

How a lead orders the backlog. Not only by "what is most valuable." Four forces:

1. **Value:** how much it contributes to the outcome (the PRD priorities).
2. **Risk and uncertainty:** do the scary, unknown things *early*, while there is time to react. A project that postpones its riskiest part to the end learns the bad news at the worst moment.
3. **Dependencies:** what must exist before something else can start.
4. **Cost:** cheap high-value items go first.

The walking skeleton. The most important early item. A **walking skeleton** (a term from Alistair Cockburn) is the **thinnest possible end-to-end implementation** that touches every architectural component and is deployed through the real pipeline. For Spendwise: the mobile app fetches the categories list from the API running in *staging*, which reads PostgreSQL, deployed by the CI pipeline, with logs and error tracking visible.

It does almost nothing useful, and it is worth a great deal, because it **proves the architecture and the plumbing work together** before you have built anything big on top. Every "it works on my machine but not in staging" surprise appears while the system is tiny and cheap to fix. Skipping it means those surprises appear in the final weeks.

Risk-first sequencing, in practice. Looking at the risk register and the contracts, the items I move *earlier* than their feature value alone would suggest are: the token-refresh logic on the mobile client (a new technology plus a subtle failure mode), the idempotent create-with-retry

flow (the product's core), and the first restore drill (the biggest recovery risk). Putting a restore drill in the *last* week is a classic mistake: if it fails, there is no time left to react.

3.4 Dependencies, the critical path, and effort versus elapsed time

A **dependency** is a relationship where one piece of work cannot start or finish until another does.

Dependency type	Example in Spendwise
Technical	The API contract before the mobile client; the migration before the code that uses the table
Data	Seeded categories before the add-expense screen can show them
Knowledge	Learning React Navigation before building the screens
Decision	The sign-in method (Q-3, ADR-005) before building authentication; future-dated expenses (Q-1) before finalising validation
External	Apple and Google developer accounts (Meera); privacy-policy text (Meera); domain; app store review time
Resource	**There is only one engineer.** Two "independent" tasks still cannot happen at the same time

The critical path is the longest chain of dependent work from start to finish; it determines the *minimum* project duration. Any delay on the critical path delays the project.

Here is the insight that surprises people on small teams:

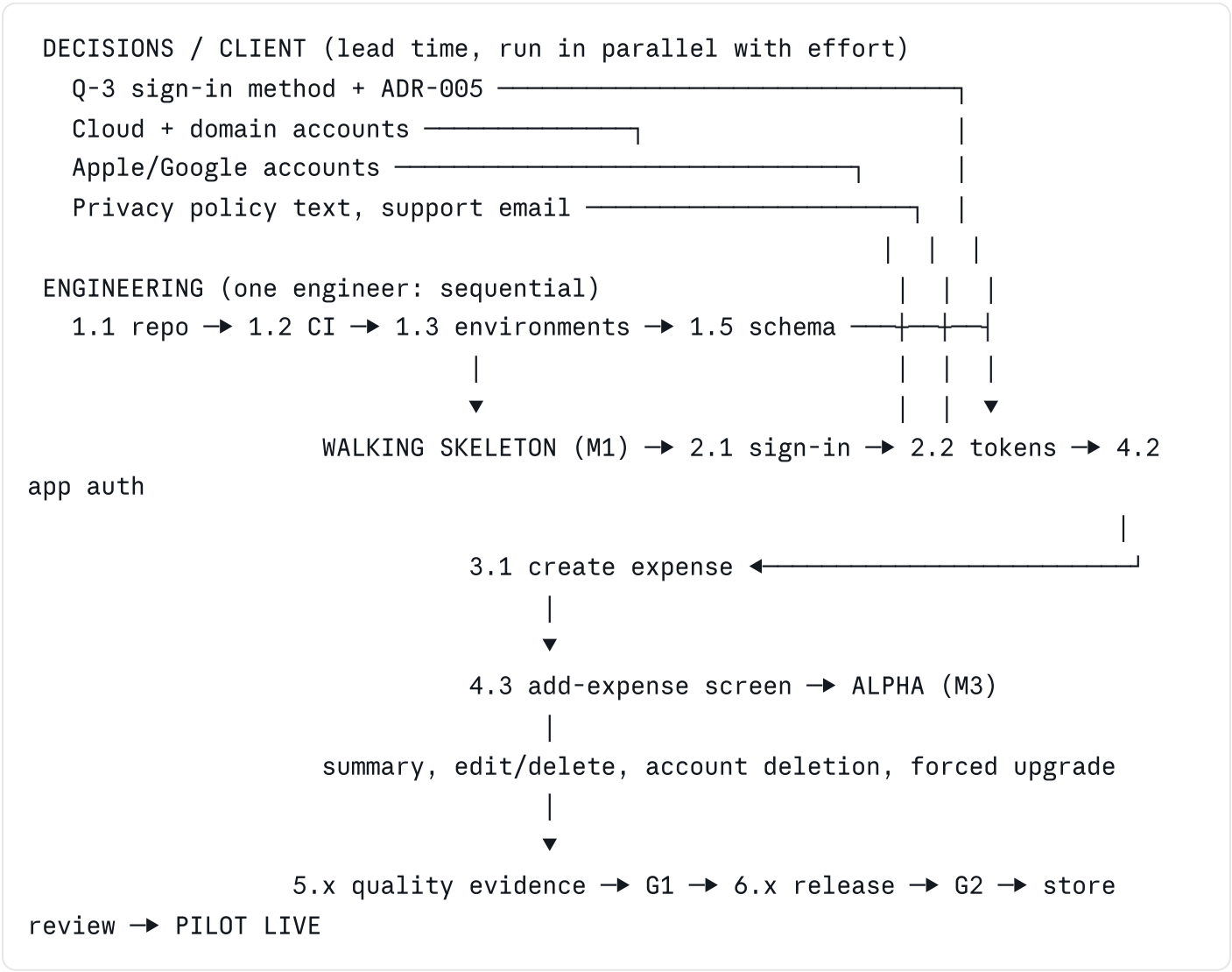
> **With one engineer, the critical path is simply all the effort, in sequence.** Because of the resource dependency, nothing really runs in parallel.

So there is one big lever for shortening *elapsed* time that does not involve working more: **items that need waiting but not effort.** App store accounts, privacy-policy text, store review, a decision from Meera, a domain registration: these take calendar time while *you* work on something else. They are the only things that can truly overlap with your effort. So a lead:

1. **Separates effort from lead time.** A task might need 2 hours of your effort but 10 days to elapse (store review). Plan both.
2. **Starts long-lead items on day one,** even though they are needed much later.

3. **Writes the client's tasks into the plan with due dates,** because "waiting for Meera" is the most common cause of slip, and it is invisible if it is not in the plan.
4. **Applies the "last responsible moment" to decisions:** for each open question, work out *by when* a decision must be made so that it does not stall the work, and tell the owner that date.

A simple dependency map for Spendwise (arrows mean "must finish before"):



(The point of the diagram is not the arrows. It is that the **left-hand boxes are not engineering,** and they have deadlines.)

3.5 Estimation

What an estimate is

An estimate is **not a number.** It is a **number with a range, a confidence, and a list of assumptions.** "About 10 hours" is a weak estimate. "Most likely 9 hours; between 6 and 16; assuming the contract is unchanged and the staging environment already works" is a good one.

Keep three quantities apart:

Quantity	Meaning	Example
Effort	Hours of focused work needed	40 hours
Capacity	Productive hours available per week	22.5 hours per week
Duration (elapsed time)	Calendar time needed	40 ÷ 22.5 ≈ 1.8 weeks

Capacity is less than availability. If you are "available" 30 hours a week, you do not do 30 hours of *productive engineering*. Client calls, planning, messages, switching between tasks, interruptions, and admin take a share. Use a **focus factor** (a fraction, often 0.6 to 0.8 for a freelancer who also handles client communication) and **write it down as an assumption.** 30 available hours × 0.75 = 22.5 productive hours.

Techniques

1. Decomposition (bottom-up). Estimate small work packages (4 to 20 hours) and add them up. This is the backbone, because small pieces are easier to reason about than large ones, and you are forced to *see* the pieces, which exposes forgotten work.

2. Three-point estimating (PERT). For each package give three numbers:

- **O**: optimistic (if everything goes well; roughly a 1-in-10 chance of doing better);
- **M**: most likely;
- **P**: pessimistic (if things go badly, but not a disaster; roughly a 1-in-10 chance of doing worse).

Then the expected value is $\mathbf{E = (O + 4M + P) \div 6}$ and a rough spread is $\mathbf{\sigma = (P - O) \div 6.}$ The formula leans toward the most likely value and gives the pessimistic tail some weight, since overruns are usually bigger than underruns. Example: O = 4, M = 8, P = 18 gives E = (4 + 32 + 18) ÷ 6 = **9.0** and σ = 14 ÷ 6 ≈ **2.3.**

Asking for **O and P** has a second benefit: it makes people *think about what could go wrong*, which is where hidden assumptions come out.

3. Analogy and historical data (reference-class forecasting). "How long did similar things take *me* before?" This is the most reliable method when you have the history, and it directly attacks the planning fallacy. At the start of a project you have no history, so you record **actuals from day one** to build it. After a few sprints you compute an **estimate accuracy ratio** (actual hours ÷ estimated hours) and apply it to the remaining estimates.

4. Spikes. If you cannot give a range (the P is unbounded because you simply don't know), do not guess: **buy information.** A spike is a time-boxed experiment (for example half a day) whose output is *knowledge* plus a better estimate. A package whose P is more than about three times its M is a candidate for a spike.

5. Relative sizing (story points, T-shirt sizes). Teams often estimate relative size (small, medium, large, or points on a Fibonacci-like scale) and use measured *velocity* to convert to time. This avoids pretending to know hours. For a **solo** developer with no history, hours with ranges are more direct, and you can switch to points once you have velocity. This is **context-dependent;** many solo developers prefer hours.

6. Planning poker and group estimation (several people estimate independently, then discuss differences) are very effective on teams, because disagreement exposes assumptions. In a solo project, the nearest equivalent is estimating twice, a day apart, or asking a peer to review your ranges.

Why the sum of uncertainty lies

Here is something you must understand, because the arithmetic invites a mistake.

If you add PERT estimates, you can also combine the spreads: for independent tasks, **variances add,** so the total $\sigma = \sqrt{(\sigma_1^2 + \sigma_2^2 + \ldots)}$. For our MVP (details in Part 4) this gives a total of about **272 hours with σ ≈ 10 hours,** which would say *"you will finish within 262 to 282 hours, about 2-in-3 odds."* That is **absurdly confident,** and it is wrong.

Why? The calculation assumes the errors are **independent** (some tasks run over, others run under, and they average out). In real projects the errors are **correlated:**

- The *same person* estimated everything, with the *same bias.*
- The *same unknown* (React Native, which you are still learning) affects *many* tasks at once.
- A *wrong design assumption* makes a whole group of tasks bigger.
- *Scope you forgot* is not in any task.

When errors are correlated, they pile up instead of cancelling. So the honest spread is far wider than the formula says. **A lead treats the arithmetic σ as a lower bound and sets the project range with judgment** (the cone of uncertainty, the learning curve, the absence of history), then replaces the guess with measured data as the project runs.

Buffers, padding, and reserves

- **Don't pad every task silently.** Hidden padding gets consumed (Parkinson's law: work expands to fill the time available) and hides the real picture. Also, "student syndrome": people start late because the task looks padded.

- **Put the reserve at the project level, visibly,** where it can be shared among tasks. (This is the idea behind *critical chain* planning.) A task that overruns draws from the reserve; one that finishes early adds to it.
- **Distinguish two reserves:** a **contingency reserve** for *known* risks and estimation error (derived from the risk register and the uncertainty), and a **management reserve** for *unknown unknowns,* which the client may hold. Tell the client what each is.
- **Never promise the optimistic number.** A range with a confidence is honest. A date with no confidence is a hope.

Estimation habits that protect you

- **Estimate from the whole WBS,** not only features (see Part 4: feature work alone is under half of the total).
- **Include test, review, debugging, deployment, and documentation** time in every package; "coding done" is not "done."
- **Separate estimating from committing** and from negotiating. Estimates are about the work; commitments are a business conversation. Do not adjust an estimate because the answer is unwelcome. Change the *scope or date* instead.
- **Beware first-time work.** For something you have never done (for you: React Native navigation, secure storage, store submission), widen P and state a **learning multiplier** as an explicit assumption.
- **Re-estimate as you learn,** and record actuals so that you can calibrate.
- **Don't let the client's number anchor you.** Estimate *first,* then compare.

Estimation for database work (your edge). You can estimate schema, migration, index, and restore tasks with unusual accuracy because you have done them. But notice that *rehearsal on production-size data* and *restore verification* are tasks, not afterthoughts, and they need a realistic dataset, which itself takes time to prepare.

3.6 Milestones

A **milestone** is a significant checkpoint with **objective, verifiable exit criteria.** It has no duration; it is a moment you can demonstrate "we are here." Milestones turn a long plan into a series of short, checkable goals.

A good milestone:

- **Is demonstrable.** You can show it working (a demo, a deployed build), not just "we feel 70% done."
- **Has exit criteria and evidence** (and, for the late ones, ties to the **release gates** from Module 5).
- **Is client-visible** where useful: Meera sees real progress, not a status report.

- **Reduces risk when reached:** each milestone retires some uncertainty (the architecture works; auth works; the core journey works; recovery works).
- **Can be planned backwards** from a target date to check feasibility.

Typical types: *technical* (walking skeleton deployed), *product* (core journey usable on a real phone), *external* (store approval), *quality* (gate G1 passed), and *go-live*.

When a milestone slips, replan visibly. The temptation is to quietly redefine the milestone ("it is basically done"). A lead does the opposite: states the slip and its cause early, and shows the revised forecast.

3.7 Sprint planning

A **sprint** is a fixed-length time box (commonly one or two weeks) in which you plan, build, demonstrate, and learn. Even for a solo engineer, time-boxing helps: it forces small scope, creates regular decision points, and produces data (velocity, accuracy) that makes your next forecast better.

Choosing the length. One week gives faster feedback but more ceremony overhead; two weeks suits a part-time solo engineer and a client who cannot review weekly. **Context-dependent.** I use **two-week sprints** here.

The sprint planning steps:

1. **Set a sprint goal:** one sentence describing the outcome, not a list of tasks. ("A user can sign up, sign in, stay signed in across token expiry, and sign out, on a real phone against staging.")
2. **Compute capacity:** available hours × focus factor, then plan to about **85%** of that. The remaining slack absorbs the unplanned (bugs from earlier work, a client question). Plan to 100% and you will fail every sprint.
3. **Pull items from the top of the backlog,** respecting dependencies, until the capacity is used. Only take items that are **ready** (clear acceptance criteria, dependencies met, estimate exists).
4. **Break items into tasks** of a few hours each. Tasks are for *you*, to see the day-to-day path; the estimate lives at the package level.
5. **Confirm the "done" definition** for each item, including tests, evidence, and deployment.
6. **Treat the plan as a forecast, not a promise.**

Sprint rituals, scaled for one person:

- **Daily plan (5 minutes):** what is the next task; what blocks it.
- **Mid-sprint check:** am I on track against the goal? If not, drop the *lowest-priority* item, do not work late.

- **Sprint review:** demonstrate the *working* result to Meera, and collect feedback. Demos of real software are the best status report.
- **Retrospective (15 minutes, written):** what slowed me, what to change, **record actual hours versus estimated.**
- **Changes mid-sprint:** a new request does not simply get added. It is **swapped** for something of equal size, or waits for the next sprint.

Carry-over rule: unfinished items do not count as partly done; they return to the backlog and are re-estimated for the *remaining* work. Counting partial credit makes velocity meaningless.

Kanban as an alternative. Instead of time boxes, you limit **work in progress** (for example, at most one or two items at a time) and measure how long items take to flow through. It suits interrupt-heavy work and very small teams. Scrum-style sprints give a stronger rhythm for client demos and forecasting. **Context-dependent;** either is acceptable if you keep the habits that matter: small items, visible priorities, regular feedback, and measured actuals.

Rolling-wave planning. You plan the *next* sprint or two in detail and the later ones coarsely, because detail about the distant future decays as you learn. The WBS and the estimate are complete (to the extent possible) up front, but the *sprint assignments* for sprints 6 to 8 are provisional.

3.8 Tracking and replanning

A plan is only useful if you compare reality against it and adjust. The tools can be simple.

- **Burn-up chart:** cumulative work completed over time, plus a line for total scope. Prefer it to a burn-down because scope changes appear as a visible step in the scope line instead of hiding.
- **Estimate accuracy ratio:** total actual hours ÷ total estimated hours for completed work. Compute it per sprint and cumulatively. If the ratio is 1.25, your remaining estimates are probably 25% low.
- **Re-forecast every sprint:** *remaining estimate* × *ratio* ÷ *capacity* gives a new completion range.
- **Track by outcome, not by "percent complete."** A package is done or not done. Never 90%.
- **Trend matters more than a single point:** a ratio that is worsening, or a milestone slipping two sprints in a row, is a signal to change something, not to hope.

The iron triangle, honestly. Scope, time, and cost (capacity) are linked, with quality as the thing you should *not* secretly trade. If the date and the capacity are fixed, scope must flex. If scope and date are fixed, capacity must grow (and adding people late has costs, as Brooks's law says: adding people to a late project can make it later because of onboarding and coordination).

Fixing all of them at once is not a plan; it is a wish. And "we'll cut testing" is not an option a lead offers, because the gates (Module 5) say what evidence is required.

Delivering bad news. The rule is **early, specific, with options.** "We will miss week 8" is a statement. "At the current estimate the full pilot needs about 16 weeks. Here are three options, and my recommendation" is leadership.

3.9 What the lead records

Artifact	Contents
WBS and dictionary	Hierarchy; package descriptions, inclusions, exclusions, done definition
Backlog	Ordered items with type, link to requirement or risk, estimate, dependencies
Estimate sheet	O, M, P, E per package; assumptions; exclusions; capacity math; date range
Dependency and decision map	Including client tasks with due dates
Plan	Milestones with exit criteria; sprint allocation; reserve
Sprint plans and retros	Goal, items, tasks, actuals
Tracking data	Planned versus actual hours; accuracy ratio; burn-up
Plan risks	New entries in the risk register (optimism, late client inputs, learning curve)
Change log	Every scope or date change, with reason and approver

Part 4 — Practical demonstration: planning the Spendwise MVP, step by step

4.1 The weak approach

```
Meera: "Launch in 8 weeks?"

Feature list from the PRD:
  Sign in ........... 3 days
  Add expense ....... 3 days
  Edit / delete ..... 2 days
  Summary ........... 3 days
  Delete account .... 1 day
  ...
Total ≈ 8 weeks.  "Yes, 8 weeks!"
```

Problems, at least ten:

1. **Only features.** No foundations, quality evidence, or release work.
2. **Task-sized in "days,"** which are not defined: calendar days or working days? Hours per day?
3. **Single numbers,** no range or confidence.
4. **No assumptions** (learning curve, availability, who decides what by when).
5. **No capacity maths:** it implicitly assumes 40 productive hours every week.
6. **No dependencies:** the sign-in method is undecided; the store accounts do not exist yet.
7. **No buffer.**
8. **No order:** nothing about doing risky things first.
9. **A date promised before a plan exists,** anchored on the client's number.
10. **No milestones, no way to track.**

4.2 Step 1: gather the inputs

PRD v0.1 (Musts and Shoulds, exclusions, open questions), the Technical Design (components, operations, failure table), the API contracts (create expense, authentication), the ERD and migration notes, the risk register v0.1 (mitigation tasks), the release gates (evidence), and constraints: **one engineer, 30 hours per week available, no history, React Native still being learned, and Meera's "8 weeks."**

4.3 Step 2: build the WBS

```
Spendwise MVP
├── 1 Foundations
|   ├── 1.1 Repository, tooling, lint/type checks, pre-commit
|   ├── 1.2 CI pipeline (unit + integration with real PostgreSQL)
|   ├── 1.3 Environments: staging + production, managed PostgreSQL, secrets,
TLS, deploy script
|   ├── 1.4 Observability baseline: structured logs, request id, error tracking,
uptime check, alerts
|   └── 1.5 Database schema v1, migrations, seed data
├── 2 Authentication and account (backend)
|   ├── 2.1 Sign-up / sign-in, hashing, throttle
|   ├── 2.2 Access + refresh tokens, rotation, reuse detection, get_current_user
|   └── 2.3 Sign-out, account deletion (two-phase), purge job
├── 3 Expenses (backend)
|   ├── 3.1 Create expense (idempotent) + contract tests        [Must]
|   ├── 3.2 Edit / delete (versioned PATCH, soft delete)         [Must]
|   ├── 3.3 Categories endpoint + seed                           [Must]
|   ├── 3.4 Monthly summary                                       [Must]
|   ├── 3.5 List with pagination                                  [Should]
|   ├── 3.6 CSV export                                            [Should]
|   └── 3.7 Usage events                                          [Must]
├── 4 Mobile app (React Native)
|   ├── 4.1 App shell: navigation, API client from OpenAPI, theme
|   ├── 4.2 Secure token storage, auth screens, single-flight refresh
|   ├── 4.3 Add-expense screen (validation, retry with same UUID, errors)
|   ├── 4.4 Monthly summary screen
|   ├── 4.5 Edit / delete expense UI
|   ├── 4.6a Sign-out and delete-account UI                       [Must]
|   ├── 4.6b Daily reminder (local notification)                  [Should]
|   ├── 4.7 Expense list screen                                    [Should]
|   └── 4.8 Error / empty / loading states + forced-upgrade check [Must]
├── 5 Quality and evidence (from risk register and gates)
|   ├── 5.1 Test hardening: two-user authorization + route meta-test,
concurrency, constraint backstops, migration tests
|   ├── 5.2 Security checks: dependency audit, secrets scan, ZAP baseline,
config review, log hygiene
|   ├── 5.3 Performance: dataset generator, load test, tuning
|   ├── 5.4 Recovery: restore drill + verification script + PITR test
|   ├── 5.5 Data-quality probes + test the alerts
|   └── 5.6 Exploratory testing, device matrix, usability test (3 people)
```

```
└── 6 Release and pilot
    ├── 6.1 Store listings, privacy disclosures, screenshots, review responses
    ├── 6.2 Mobile build/signing/release pipeline, test tracks
    ├── 6.3 Runbook, README, incident procedure, second-person dry run
    └── 6.4 Pilot onboarding, feedback channel, first-week monitoring
```

Check against the 100% rule and the forgotten-categories list: foundations present; quality *derived from the risk register and gates* (5.x); release plumbing present (6.x); client tasks tracked separately (below). Excluded on purpose (recorded): bank import, offline, shared households, custom categories, charts, dark mode, receipt scanning (the PRD exclusions); hosting costs and store fees (client's budget); design assets beyond a simple theme.

4.4 Step 3: estimate with ranges and assumptions

All values in **focused hours.** O, M, P as defined; E = (O + 4M + P) ÷ 6; "Plan h" is E rounded to whole hours (so the column adds up).

ID	Work package	Pri	O	M	P	E	Plan h
1.1	Repo, tooling, checks	M	3	4	8	4.5	5
1.2	CI pipeline	M	4	6	12	6.7	7
1.3	Environments, hosting, secrets, TLS, deploy	M	6	10	20	11.0	11
1.4	Observability baseline	M	4	6	12	6.7	7
1.5	Schema v1, migrations, seed	M	3	5	8	5.2	5
2.1	Sign-up / sign-in, hashing, throttle	M	8	12	22	13.0	13
2.2	Tokens, rotation, reuse detection	M	10	16	30	17.3	17
2.3	Sign-out, deletion, purge job	M	6	9	16	9.7	10
3.1	Create expense (idempotent)	M	6	9	16	9.7	10
3.2	Edit / delete	M	6	9	15	9.5	10
3.3	Categories	M	2	2	4	2.3	2
3.4	Monthly summary	M	3	5	9	5.3	5
3.7	Usage events	M	2	4	7	4.2	4
4.1	App shell, navigation, API client	M	6	10	20	11.0	11
4.2	Secure storage, auth screens, single-flight refresh	M	10	18	34	19.3	19
4.3	Add-expense screen	M	9	14	26	15.2	15
4.4	Summary screen	M	5	8	14	8.5	9
4.5	Edit / delete UI	M	5	8	14	8.5	9
4.6a	Sign-out, delete-account UI	M	3	5	9	5.3	5
4.8	States + forced-upgrade check	M	4	7	12	7.3	7
5.1	Test hardening	M	8	12	22	13.0	13
5.2	Security checks	M	4	6	12	6.7	7

ID	Work package	Pri	O	M	P	E	Plan h
5.3	Performance work	M	6	10	20	11.0	11
5.4	Recovery drill + scripts	M	6	10	20	11.0	11
5.5	Probes + alert tests	M	3	5	9	5.3	5
5.6	Exploratory, devices, usability	M	5	8	14	8.5	9
6.1	Store listings and disclosures	M	6	10	18	10.7	11
6.2	Mobile release pipeline	M	5	8	16	8.8	9
6.3	Runbook, README, dry run	M	5	8	14	8.5	9
6.4	Pilot onboarding, monitoring	M	3	6	10	6.2	6
	Total, Musts						**272**
3.5	List with pagination	S	5	8	14	8.5	9
3.6	CSV export	S	3	5	9	5.3	5
4.6b	Daily reminder	S	5	8	14	8.5	9
4.7	List screen	S	5	8	14	8.5	9
	Total, Shoulds						**32**

Subtotals of the Musts: Foundations 35 · Auth 40 · Expenses 31 · Mobile 75 · Quality 56 · Release 35 = **272 hours.**

Estimate assumptions (written next to the numbers):

1. Estimates are for the one engineer, in focused hours, and include self-review, debugging, tests, deployment, and documentation for each package.
2. **React Native is still being learned;** the P values for mobile packages include a learning allowance, and this is the largest source of uncertainty.
3. The architecture and contracts do not change materially. A change in the sign-in decision (ADR-005) changes packages 2.x and 4.2.
4. Managed PostgreSQL and a PaaS or managed container service are used, with no custom infrastructure.
5. A simple default theme and component library; no custom design or illustration work.

6. Meera replies to questions within two working days, and the client tasks below are delivered by their due dates.
7. No password-reset flow (open question Q-6); adding it would add roughly 20 hours (a rough guess to be re-estimated if it is added).
8. Excluded: ongoing support after launch, hosting and store fees, translation, accessibility audit beyond basics.

What the feature-only view would have said. Auth (40) + Expenses (31) + Mobile (75) = **146 hours.** The other **126 hours (46%)** are foundations, quality evidence, and release work. At 22.5 productive hours per week, 146 hours is about 6.5 weeks, which is very close to Meera's "8 weeks." *That is exactly how a plan built only from features produces an attractive and false answer.*

4.5 Step 4: look honestly at the uncertainty

The arithmetic spread. Summing the variances of the Must packages ($\sigma = (P - O) \div 6$ for each, squared, added) gives about **105.6, so $\sigma \approx$ 10.3 hours.** The formula claims 272 ± 10, within about 4%. A lead recognises this as **wrong,** because the errors are correlated (one estimator, one unfamiliar framework, no history). So I set the planning range by judgment, stating the reasons:

Level	Hours	Basis
Raw sum (best-effort "if my estimates are right")	**272**	PERT expected values
Expected	**~313** (+15%)	First estimates by one person with no history are usually optimistic; some work is always missing
Commit-level (a date I'd be comfortable promising)	**~367** (+35%)	Cone of uncertainty at this stage, React Native learning curve, correlated errors

These multipliers are **explicit assumptions,** not facts. After the first sprints, **measured actuals replace them.**

4.6 Step 5: capacity and dates

- **Availability:** 30 hours per week (client work, planning, and calls included in this).
- **Focus factor:** 0.75 → **22.5 productive hours per week.**
- **A two-week sprint:** 45 productive hours; **plan to about 85% → about 38 hours.**

Scenario	Hours	÷ 22.5 per week	Weeks
Raw (best case)	272	12.1	**~12**
Expected	313	13.9	**~14**
Commit-level	367	16.3	**~16**

The sprint plan (below) uses 38-hour sprints, and 272 ÷ 38 = 7.2, so **8 two-week sprints = 16 weeks,** which matches the commit-level date. The 304 hours of sprint capacity exceed the 272 hours of Musts by 32 hours, which is the **reserve.** The four Shoulds (also 32 hours) are **not scheduled;** they enter the plan only if the reserve is still unspent after Sprint 6.

4.7 Step 6: reconcile with Meera's "8 weeks"

First, what 8 weeks can hold: at 22.5 productive hours per week, 8 weeks is 180 productive hours; planned at 85% that is about **152 plannable hours,** against **272 raw hours** of Musts. The gap is 120 hours, or **44% of the work.** There is no honest way to remove 44% of the work and still pass the gates in Module 5.

The options, with their trade-offs (and why each is or is not recommended):

Option	Effect	Trade-off	Verdict
A. Keep scope, move the pilot date	Public pilot ≈ week 16 (commit) with a range of 12 to 16	Disappoints an eager audience; honest	**Recommended, with F**
B. Cut scope to fit 8 weeks	Impossible without cutting Musts or the evidence the gates require	Cutting quality evidence silently is the dangerous trade	Rejected
C. Add capacity: 40 h/week	30 productive h/week → raw 9.1 weeks, commit ≈ 12.2 weeks	+33% cost; burnout; **still not 8 weeks**	Possible if Meera funds it; does not reach 8
D. Add a second developer for mobile	Might reach about 11 to 12 weeks	Onboarding, coordination overhead, cost, new risk (Brooks's law); contract-first helps	Consider only if the date is truly hard
E. Use a managed identity provider (ADR-005 Option B)	Roughly −22 hours (about 8%): backend sign-in/token packages ≈ 30 h replaced by ≈ 15 h of provider integration; mobile auth saves ≈ 7 h	Recurring cost, lock-in, residency terms; saves about one week	A real input to ADR-005, but not decisive on its own
F. Phase the delivery	**Alpha at week 8** (sign-in + add expense on a real phone, for Meera and 2 to 3 friends), pilot at week 16, Shoulds after	Alpha is deliberately incomplete; needs a clear message	**Recommended with A**

Why an alpha at week 8 makes sense: the product's riskiest assumption is that logging an expense is **fast enough** that people do it daily. The add-expense journey is exactly what the alpha tests, with real people, early, while changes are still cheap. That is risk-first sequencing, and it gives Meera something real to show her friends at about the time she hoped.

What I'd tell Meera (a draft, in the spirit of "what I know, what I assume, what I need"):

> *"I've broken the work down and estimated it, including the parts that don't show up on screen: environments, security checks, backup and recovery tests, and app store preparation, which together are almost half the effort. At 30 hours a week, the full pilot is most likely 14 weeks of work, and I'd commit to week 16. Eight weeks is not realistic for the full pilot without skipping safety work I don't want to skip on a finance app. What I can do by*

> *week 8 is an alpha you and two or three friends can use on your phones to add expenses, so we learn early whether logging is fast enough. I'd also like your help on four dated items (accounts, a decision on how people sign in, and privacy policy text), listed below. I'll update the forecast after each sprint using real numbers. Can we talk through the options on a call this week?"*

4.8 Step 7: dependencies, decisions, and client tasks with due dates

Using the "last responsible moment" for each, working back from where the work needs it:

Item	Owner	Needed by	Why then	Risk if late
Cloud, domain, and hosting accounts created	Meera (with lead)	End of week 1	Package 1.3	Blocks the walking skeleton
Decision: sign-in method (Q-3) and ADR-005	Meera + Lead	**Before Sprint 2 (week 3)**	Package 2.1 builds either own auth or a provider integration	Rework of ≈ 30 hours
Q-2: fixed category list approved	Meera	Before 3.3 (Sprint 2)	Seeding	Small rework
Apple Developer and Google Play accounts	Meera	End of week 4 (start in week 1)	Package 6.2 test tracks; enrollment time can vary from days to weeks	Store risk R-08; schedule slip
Q-1: future-dated expenses rule	Meera	Before 3.1 (Sprint 4, week 7)	Validation rule	Rework of validation and UI
Q-6: password reset needed?	Meera	**Before Sprint 3 (week 5)**	Adds ≈ 20 hours if yes; affects the date	Late scope addition
Privacy policy and support email text	Meera (legal check)	Week 10	Package 6.1 and 4.6a	Blocks store submission
Pilot user list and consent	Meera	Week 12	6.4	Delays onboarding
Second person for the runbook dry run	Meera	Week 14	6.3, gate G2	Gate cannot pass

Notice that **four of these are due in the first five weeks,** when the engineering work looks unrelated to them. That is why they must be in the plan, with owners, from the beginning.

4.9 Step 8: sequence the work, draw the sprints, set milestones

Risk-first ordering gives this sprint allocation (hours are Plan h; each sprint targets about 38):

Sprint	Weeks	Goal	Items	Hours
S1	1–2	Foundations ready	1.1, 1.2, 1.3, 1.4, 1.5	35
S2	3–4	**Walking skeleton** deployed	3.3, 4.1, 2.1, 3.7	30
S3	5–6	**Secure sign-in on a real phone** (highest technical risk)	2.2, 4.2	36
S4	7–8	**Core journey (alpha):** add expense, retry-safe, plus the **first restore drill**	3.1, 4.3, 5.4	36
S5	9–10	See where money went; fix mistakes	3.4, 4.4, 3.2, 4.5	33
S6	11–12	Feature-complete Musts; early store test build	2.3, 4.6a, 4.8, 5.2, 6.2	38
S7	13–14	**Evidence for gate G1**	5.1, 5.3, 5.5, 5.6	38
S8	15–16	**Go-live gate G2,** submission, runbook	6.3, 6.1, 6.4 (+ the re-drill and rollback drill in the slack)	26
			Total	**272**

(If Meera's store accounts arrive early, swap 6.2 with 4.5 so that a test build reaches the stores sooner, which reduces the store-review risk.)

Milestones with exit criteria:

Milestone	End of	Exit criteria (all verifiable)	Evidence
M0 Plan agreed	Week 1	WBS, estimate, assumptions, and date range reviewed with Meera; decisions list with owners and dates accepted	Written agreement
M1 Walking skeleton	Week 4	CI green; `main` deploys to staging by pipeline; migrations run by pipeline; the app on a real phone fetches categories from staging; logs and error tracking visible	Demo + CI link
M2 Secure sign-in	Week 6	Sign-up, sign-in, refresh, sign-out work on a real phone; reuse-detection and throttle tests pass	Demo + CI report
M3 Alpha (core journey)	Week 8	Add expense works on a real phone with retry safety; first restore drill logged with measured time; Meera + 2 to 3 friends use it; time-to-log measured	Demo, drill log, feedback notes
M4 Feature complete	Week 12	All Musts working; account deletion verified; forced-upgrade check works; internal test build on both stores' test tracks	Checklist, builds
M5 Gate G1 passed	Week 14	All G1 criteria with evidence tied to the release commit (Module 5)	Evidence pack
M6 Gate G2 passed + submission	Week 16	Restore drill within RTO; rollback drill; dry run; client acceptance of residual risks; submitted for review	Evidence pack, acceptance
M7 Pilot live + G3	Week 16 to 17 (store review)	Production smoke tests pass; 48-hour watch clean	Dashboards, probe history

4.10 Step 9: plan Sprint 3 in detail (the shape of a sprint plan)

Sprint 3 (weeks 5–6). Capacity: 45 productive hours; plan about 38. Planned: **36** hours. **Goal:** *A user can sign up, sign in, stay signed in across access-token expiry, and sign out on a real phone against staging.* **Ready check:** ADR-005 decided (own implementation), Q-6 answered, auth contract v0.1 approved, `refresh_tokens` migration drafted. ✓

Item	Task	Hours
2.2 Tokens, rotation, reuse detection (17 h)	Migration for `refresh_tokens` and constraints	2
	Token generation and hashing	2
	Sign-in issues a session and token pair	2
	Refresh endpoint: row lock, rotation, grace window	4
	Reuse detection and commit-before-respond	3
	`get_current_user` verification order and claims	2
	Contract tests for the above	2
4.2 Secure storage, auth screens, single-flight refresh (19 h)	Secure storage module (Keychain / Keystore)	3
	API client with auth header and refresh interceptor	4
	Single-flight refresh (one refresh at a time)	4
	Sign-up and sign-in screens with validation	5
	Map error codes to UI actions	3

Definition of done for every task: code reviewed against the checklist; tests written and passing in CI; contract file updated if behaviour changed; deployed to staging; evidence recorded (CI link; short demo clip for Meera).

Slack: about 9 hours (45 − 36) for unplanned work. **Review:** demo on a real phone to Meera at the end of week 6. **Retro:** record actual hours per package.

4.11 Step 10: track and replan, using real numbers

A short illustration (all numbers invented for teaching) of what happens after Sprint 3 when the estimate meets reality:

Sprint	Planned (estimate-hours completed)	Actual hours spent	Accuracy ratio
S1	35	41	1.17
S2	30	37	1.23
S3	36	49	1.36
Cumulative	**101**	**127**	**1.26**

(Sprint 3's package 4.2 took about 31 hours against a 19-hour estimate: the React Native learning curve showed up exactly where the largest uncertainty was predicted.)

Re-forecast: remaining raw estimate = 272 − 101 = **171 hours.** Calibrated by the cumulative ratio: 171 × 1.26 ≈ **215 actual hours.** At about 38 plannable actual hours per sprint, that is **5.7 more sprints → 6 more,** so **9 sprints in total = 18 weeks,** two weeks later than the 16-week commit.

What a lead does with that:

1. **Tells Meera now,** with the data and the options (the Shoulds were never scheduled; the reserve is consumed; do we move the date by two weeks or re-sequence?).
2. **Looks for the cause, not just the number.** The overrun is concentrated in mobile (React Native). A better forecast applies a *higher ratio to remaining mobile packages* (about 45 raw hours remain: 4.3, 4.4, 4.5, 4.6a, 4.8) and a *lower ratio to backend packages* (which are running close to estimate).
3. **Considers a spike or targeted help,** such as a half-day review with an experienced React Native developer, to attack the biggest source of overrun.
4. **Updates the risk register** (the learning-curve risk is now observed: likelihood 5).
5. **Does not hide it in overtime.** Working 45 hours a week to protect the date hurts quality, hides the true ratio, and teaches the client the wrong lesson.

Part 5 — Documentation: what the lead records, and how much

Artifact	Purpose	Detail level	Update rule
WBS + dictionary	Defines the scope of work	One page for the tree; a few lines per package	When scope changes (logged)
Backlog	Ordered list of items	One line per item with ID, type, link, estimate, dependencies	Continuously; refined each sprint
Estimate sheet	O, M, P, assumptions, exclusions, capacity math	One table + one assumptions list	Re-forecast each sprint
Dependency and decision map	Visible waits, owners, due dates	One table	Weekly
Plan	Milestones, sprint allocation, reserve	One page	Each sprint
Sprint plan + retro	Goal, items, tasks, actuals, lessons	Half a page each	Each sprint
Tracking data	Planned vs actual; accuracy ratio; burn-up	A simple table or chart	Each sprint
Change log	Scope or date changes: what, why, who approved, effect	One line per change	At every change
Plan risks	In the risk register	Standard format	With the register

Practices that keep the plan honest:

- **Estimate sheet and assumptions travel together.** If an assumption changes, the estimate is stale.
- **Record actuals every sprint,** even approximately. Without them, you cannot calibrate.
- **Keep the plan in the repository** (a Markdown file), next to the code and the risk register, so it is versioned and findable.
- **Share the plan with the client,** in plain language, and explicitly list *their* tasks.
- **Always attach a confidence to a date.**

Anti-patterns:

- **Gantt chart theatre:** a beautiful chart that nobody updates.
- **False precision:** "13.4 weeks." Round honestly and give a range.
- **A single number with no assumptions.**
- **Estimating only the features.**
- **Planning at 100% of capacity.**
- **The plan that never changes** (or changes silently).
- **Padding every task secretly.**
- **Letting the client's number become your estimate.**
- **Counting partial work as done.**

Part 6 — Trade-offs and failure scenarios

I mark each **[Established]** (widely agreed) or **[Context]** (judgment).

6.1 Fixed scope, fixed date [Established]

If scope, date, and capacity are all fixed, something has to give, and it is usually quality (tests, recovery drills, security) because it is the least visible. A lead names the trade explicitly and refuses to cut safety work silently.

6.2 Feature-only estimate [Established]

Scenario. The estimate covers screens and endpoints. Three weeks before launch, the work for CI, environments, monitoring, store submission, restore drills, and load tests appears, all at once, and the date slips by two months. **Prevention:** derive the WBS from the technical design, the risk register, and the gates; compare "feature-only" with "whole" explicitly.

6.3 Estimating what you don't understand [Established]

A package whose pessimistic estimate is several times its most likely is telling you that you lack information. **Prevention:** a spike (time-boxed learning) before estimating, not a bigger guess.

6.4 Summing uncertainty as if it were independent [Established]

The arithmetic says ±4%, reality says ±35% or more. **Prevention:** treat the formula's spread as a lower bound; set the range using judgment and the cone of uncertainty; replace it with measured ratios as soon as you have actuals.

6.5 Hidden padding vs a visible reserve [Context]

Padding each task hides information and gets consumed. A visible, shared, project-level reserve is more honest and more efficient. Some teams prefer rounding estimates up and holding a small reserve; the point is *visibility*.

6.6 Story points vs hours [Context]

Points avoid false precision and let velocity do the conversion, and are useful for teams with history. Hours with ranges are clearer for a solo developer at the start. Either works if you track actuals.

6.7 Scrum vs Kanban for one person [Context]

Sprints give rhythm, demos, and forecasting. Kanban with work-in-progress limits suits interrupt-heavy work. Pick one deliberately; don't mix the overhead of both.

6.8 Sprint length [Context]

One week: fast feedback, more overhead. Two weeks: suits part-time work and a client who reviews every two weeks. Four weeks or more: too long to correct course.

6.9 Adding people to a late project [Established]

Brooks's law: adding people to a late software project tends to make it later (onboarding, communication, splitting work). It can help early, with clean contracts and separable work, such as a mobile developer working against a mock of the API. It rarely rescues a plan late in the project.

6.10 Overcommitting sprints [Established]

Planning to 100% of capacity means every unplanned bug or call causes a miss. Repeated misses destroy trust in the plan. Plan to about 85%.

6.11 Gold-plating and Parkinson's law [Established]

Work expands to fill the time available. Without time boxes and a clear "done," packages grow polish nobody asked for.

6.12 Quality squeezed at the end [Established]

Scenario. Features run long. The final two weeks, which held the load test, the restore drill, and the runbook, are squeezed to three days. The drill is skipped. **Prevention:** schedule risk-driven work *early* (the restore drill in Sprint 4), treat the gates as scope, and protect the evidence sprints from absorbing feature overflow.

6.13 Unmanaged external dependencies [Established]

Scenario. The Apple developer account is requested in week 12, and enrollment takes longer than expected. The app cannot be submitted. **Prevention:** start long-lead items on day one, track them with owners and due dates, and add the risk to the register.

6.14 Context switching and the focus factor [Context]

If you serve several clients or have a day job, 30 "available" hours may yield only 15 productive ones. Estimate the focus factor honestly; if unsure, measure it for two sprints.

6.15 Heroic overtime [Established]

Overtime hides the true ratio, causes defects, and burns out the only engineer (itself a registered risk, the bus factor). Use data to renegotiate scope or date instead.

6.16 Big-bang release vs phased delivery [Context]

A single launch maximises surprise at the end. Phased delivery (alpha, pilot, public) reduces risk and shortens feedback loops, but repeats some release overhead (each release has its own submission, watch period, and checks). Here, the benefit outweighs the repeated cost.

6.17 Over-planning [Context]

Detailed plans for sprints six months away are fiction. Plan the next two sprints in detail, the rest coarsely, and refine as you learn (rolling-wave planning).

6.18 The "#NoEstimates" perspective [Context]

Some teams argue that if work is split into small, similarly sized items, counting items replaces estimating hours. It works with stable teams and history. For a client who needs a date range at the start, estimates are still needed, but the idea reinforces *small items and measured throughput.*

Part 7 — Hands-on exercise

You are the lead. The MVP is live. Now plan the next release.

Situation

Release 1.2: the pilot users have asked for two things (from Lecture 4): *"I changed my password, so please sign out my old tablet,"* and *"show me where I'm signed in."* The requirements:

- **R-1:** A signed-in user can change their password. After the change, **all other devices are signed out;** the current device stays signed in (it receives fresh tokens).
- **R-2:** A signed-in user can see their sessions (device name, start time, last used) and sign any one of them out.
- **R-3:** At most 10 active sessions per user; signing in on an 11th device ends the oldest.

Meera: *"Can you do this in three weeks? We're launching a public sign-up page right after."*

Facts you have

- **Capacity:** you can give the project **20 hours a week,** with a **focus factor of 0.75,** and you plan sprints to **85%.**
- **History from the MVP build:** estimate accuracy ratio (actual ÷ estimate) was **1.1 for backend packages, 1.45 for mobile packages, and 1.2 for everything else.**

- **Technical context:** the `refresh_tokens` table now holds about **1.5 million rows.** The design (Lecture 4) introduces a `sessions` table for R-2 and R-3 with a batched backfill and a two-phase foreign key; **R-1 alone needs no new table** (it revokes all existing sessions and issues a new one).
- **Release rules from Module 5:** the release still needs security checks, a performance sanity run, a rollback drill with an updated runbook, a store submission, and a 48-hour post-release watch. The store review takes between 1 and 7 days.
- **Dependencies:** a **second device** is needed for testing; a **production-size copy** of the data is needed to rehearse the backfill; Meera must be available for a **dry run** of the runbook.

The work packages (not yet estimated; you must estimate them)

Backend

- B1 Password-change endpoint (verify current password, hash, one transaction: update hash and `password_changed_at`, revoke sessions, create a new session)
- B2 `sessions` table, dual-write on sign-in, session-limit enforcement under a user-row lock
- B3 Sessions list and revoke endpoints, contract and OpenAPI
- B4 Backfill sessions for existing token families in batches; add the foreign key `NOT VALID`, then validate; scripts, verification, and a rehearsal on the production-size copy
- B5 Tests: contract tests, concurrency tests (simultaneous password changes; 11 simultaneous sign-ins), authorization tests, backfill idempotency

Mobile

- M1 Change-password screen, token replacement from the response, error mapping (403 vs 401)
- M2 Sessions screen (list, revoke, current-device marker)
- M3 Handling `token_revoked` and `invalid_refresh_token`: wipe local data, explanatory UI
- M4 Two-device testing on real phones

Quality and release

- Q1 Security checks refresh (dependency audit, secrets scan, log hygiene including passwords)
- Q2 Performance sanity on sign-in and refresh with the new table; pool check
- Q3 Rollback drill and runbook update (including migration steps)
- R1 Store build, submission, release notes
- R2 Post-release 48-hour watch and verification

Tasks (about 2 hours)

1. **WBS (15 min).** Produce a WBS with these packages, checking the 100% rule and the forgotten-categories list. Add anything missing and say why.
2. **Estimate (30 min).** Give **O, M, P** for each package, compute **E** (PERT) and round. Write at least **five assumptions.**
3. **Calibrate (15 min).** Apply the historical ratios appropriately (backend, mobile, other). Show the arithmetic. Explain why you use the ratios rather than just the raw estimates.
4. **Capacity and date (15 min).** Compute weekly plannable hours, and the duration for (a) the **full scope,** and (b) a **reduced scope** of your choice. Show the maths.
5. **Dependencies and critical path (15 min).** List dependencies (technical, external, decision, resource), identify what has **lead time** versus effort, and say what you start on day one. State the earliest **go-live** date including store review.
6. **Options and message (20 min).** Present at least three options to Meera (scope, capacity, date, phasing), with trade-offs and a recommendation, and write a **short client message** (what you know, assume, and need).
7. **Milestones and sprint plan (15 min).** Define milestones with exit criteria, and a week-by-week (or sprint-by-sprint) allocation of hours for your recommended option.
8. **Risks created by the plan (5 min).** Add at least three new risks (cause → event → impact), with owners and early warnings.

What I'll look for (so you know the standard)

- A WBS that includes *non-feature* work (quality, release, dependencies), not only the screens and endpoints.
- Ranges and assumptions, not single numbers; correct PERT arithmetic.
- **Calibration with the historical ratios,** applied to the right categories.
- Capacity math with the focus factor and the 85% rule; duration computed from calibrated hours.
- Dependencies with **lead time** separated from effort, and an honest earliest date including store review.
- Options with real trade-offs, a clear recommendation, and a client message that gives bad news early, specifically, and with options.
- Milestones that are verifiable; a plan that fits the capacity.
- No silent cutting of the safety and evidence work.

Part 8 — Review of a typical attempt

As before, since this lecture is one-way, I review the kind of answer a learner at your level commonly writes: hints first, then corrections.

8.1 A typical first attempt (call the learner "Sam")

> **WBS:** Design, Code, Test, Release. **Estimates:** Password change: 3 days. Sessions list: 2 days. Backfill: 1 day. Mobile screens: 3 days. Testing: 2 days. Release: 1 day. Total: 12 days = 2.4 weeks. **Capacity:** 20 hours a week. **Plan:** "Everything fits in three weeks. If we run late I'll work extra hours." **Dependencies:** "Backend first, then mobile." **Milestones:** "Release on day 15." **Message to Meera:** "Yes, three weeks!"

8.2 The review

What Sam did well.

- Sam included mobile and release work, not only the backend.
- Sam noticed a sensible build order (backend, then mobile).
- Sam named a capacity figure instead of silently assuming full time.

Gap 1: the WBS is phases, not deliverables. "Design, Code, Test, Release" hides everything and cannot be tracked. *Hint:* Can you say, for each box, *what exists when it is done? Correction:* deliverable-based packages with a "done" definition (see the reference).

Gap 2: single numbers in "days." Days of what? Eight hours? And no range, no assumptions. *Hint:* What would make "backfill: 1 day" 3 days? What are you assuming about the data copy? *Correction:* O/M/P in focused hours, E by PERT, with assumptions.

Gap 3: the capacity arithmetic is wrong. 12 days of 8 hours is 96 hours, but Sam has 20 hours a week; 96 hours is nearly five weeks of *nominal* time, before the focus factor. "2.4 weeks" came from dividing days by 5 as if working full time. *Hint:* How many hours does Sam actually have in three weeks? What fraction is productive? *Correction:* 20 × 0.75 = 15 productive hours per week; at 85% about 12.75 plannable.

Gap 4: no calibration. The history says mobile work ran 1.45× over. Sam's mobile estimate takes none of that into account. *Hint:* What does the ratio tell you about *your* estimates, specifically for mobile? *Correction:* multiply remaining mobile estimates by about 1.45, backend by 1.1, others by 1.2.

Gap 5: "I'll work extra hours." That hides the real ratio, hurts quality, and the only engineer is also a risk (the bus factor). *Hint:* What would you tell Meera if you were already at your limit? *Correction:* change scope, date, or capacity openly.

Gap 6: dependencies ignore lead times. Store review (1 to 7 days), the second device, the production-size copy, and Meera's dry-run availability are not mentioned. The "release on day 15" ignores the review wait. *Hint:* Which of these take calendar time but not effort? What do you start first? *Correction:* a dependency table with lead times and a first-day action list.

Gap 7: no options, a premature yes. The honest answer, three weeks for everything, is not available. Saying yes anchors on the client's number. *Hint:* What would you give Meera in three weeks that is *honest? Correction:* phase the release (password change first), show the maths, offer alternatives.

Gap 8: milestones are only the end. No intermediate checkpoint, no exit criteria, no evidence. *Hint:* What would show, in week 2, that you are on or off track?

Overall verdict. Sam produced a hopeful list, not a plan. The habit to build: **decompose into deliverables, estimate in ranges with assumptions, calibrate with history, convert to dates through real capacity, and present options, with lead times on every dependency.**

Part 8B — Reference solution to the exercise

Attempt the exercise first.

1. WBS (with additions)

```
Release 1.2
├── Backend
|   ├── B1 Password change (transaction, revocation, new session)
|   ├── B2 sessions table + dual-write + session limit under lock
|   ├── B3 sessions list + revoke + contract/OpenAPI
|   ├── B4 backfill + FK (NOT VALID → VALIDATE) + rehearsal on prod-size copy
|   └── B5 tests: contract, concurrency, authorization, backfill idempotency
├── Mobile
|   ├── M1 change-password screen + token replacement + error mapping
|   ├── M2 sessions screen
|   ├── M3 token_revoked / invalid_refresh_token handling + wipe + copy
|   └── M4 two-device testing
├── Quality and release
|   ├── Q1 security checks refresh
|   ├── Q2 performance sanity (sign-in/refresh with new table; pool)
|   ├── Q3 rollback drill + runbook update
|   ├── R1 store build, submission, release notes
|   └── R2 48-hour watch + verification
└── Client / dependency tasks (not engineering effort)
    ├── Second device available for M4
    ├── Production-size data copy provisioned (for B4 rehearsal)  [engineering,
but needs lead time]
    ├── Meera's availability for the runbook dry run
    └── Store review (1–7 days)
```

Additions I would make to the given list: the production-size copy as an explicit task (it takes time to create and anonymise); a **release-notes and user communication** item (inside R1); and a **decision** item: confirm with Meera that existing sessions will be signed out when the release changes token handling, if any user-visible effect exists. The packages cover the Module 5 gate evidence (security, performance, rollback, watch).

2. Estimates (focused hours)

ID	Package	O	M	P	E	Plan h
B1	Password change	5	8	14	8.5	9
B2	Sessions table, dual-write, limit	5	8	14	8.5	9
B3	Sessions list/revoke + contract	3	5	9	5.3	5
B4	Backfill, FK validation, rehearsal	5	8	15	8.7	9
B5	Tests	6	10	18	10.7	11
M1	Change-password screen	4	7	13	7.5	8
M2	Sessions screen	4	7	13	7.5	8
M3	Revoked-token handling	3	5	9	5.3	5
M4	Two-device testing	2	4	7	4.2	4
Q1	Security checks	2	3	5	3.2	3
Q2	Performance sanity	3	5	9	5.3	5
Q3	Rollback drill + runbook	2	4	7	4.2	4
R1	Store build and submission	2	4	8	4.3	4
R2	48-hour watch	2	3	5	3.2	3
	Total (full scope)					**87**

By category: backend 43 · mobile 25 · quality and release 19.

Assumptions (examples): (1) the Lecture 4 design is accepted unchanged; (2) staging and a pipeline already exist from the MVP; (3) a production-size anonymised copy can be created in about two hours; (4) existing token-handling code in the app is reused, so M1 and M3 are not

first-time React Native work; (5) store review takes 1 to 7 days; (6) Meera can do a one-hour dry run in the final week; (7) no change to the password policy.

3. Calibration

Category	Raw hours	Ratio	Calibrated
Backend (B1 to B5)	43	1.1	47.3
Mobile (M1 to M4)	25	1.45	36.3
Quality and release (Q, R)	19	1.2	22.8
Total	**87**		**106.4**

Why use the ratios: because history is the best defence against the planning fallacy. The raw 87 hours is what *I believe;* 106 hours is what *my own past* says I should expect. The mobile ratio is highest because that was where the learning curve bit; if this release's mobile work genuinely reuses known patterns, I might use a lower mobile ratio, but I would state that as an explicit assumption and not simply assume it.

4. Capacity and dates

- Available: 20 h/week; focus factor 0.75 → **15 productive h/week;** at 85% → **12.75 plannable h/week.**
- **Full scope:** 106.4 ÷ 12.75 = **8.3 weeks** (before store review).
- **Reduced scope, "1.2a": password change only (R-1).** Packages: B1 (9), a reduced B5 (6: contract, concurrency, authorization for the password change), M1 (8), M3 (5), M4 (4), Q1 (3), Q3 (3, reduced), R1 (4), R2 (3). Raw total **45.**

Category	Raw	Ratio	Calibrated
Backend (B1 9 + B5-lite 6)	15	1.1	16.5
Mobile (M1 8 + M3 5 + M4 4)	17	1.45	24.65
Quality and release (Q1 3 + Q3 3 + R1 4 + R2 3)	13	1.2	15.6
Total	**45**		**56.75**

56.75 ÷ 12.75 = **4.45 weeks** of work, so about **5 weeks,** plus store review.

5. Dependencies, lead times, critical path

Dependency	Type	Lead time	Action
Second device for M4	Resource	Days (borrow or buy)	Arrange in **week 1**
Production-size copy (for B4, release 1.2b)	Technical	~Half a day of effort, plus provisioning	Schedule before B4 starts
Meera's dry-run availability	External	Needs a booked slot	Book in **week 1** for the final week
Store review	External	**1 to 7 days elapsed**	Submit as early as the build is ready
Sign-out behaviour decision	Decision	A conversation	Confirm in week 1
Backend before mobile (contract-first mock helps)	Technical		Use the contract to start M1 against a mock

Critical path (one engineer): all the effort in sequence, because of the resource constraint. The only real parallelism is the *waiting* items: the device, the booking, the copy, and the store review. **Earliest go-live:** about 5 weeks of work plus up to a week of store review: **week 6** at the planned capacity (and the 48-hour watch after).

6. Options and the message to Meera

Option	Effect	Trade-off	Verdict
A. 1.2a now, 1.2b later (phase)	R-1 live by ~week 6; R-2 and R-3 follow	Repeats ~7 hours of release overhead (second submission and watch); two releases	**Recommended**
B. All three in one release	~8.3 weeks of work + review → ~week 9 to 10	Later value; larger release, larger blast radius	Not recommended
C. Raise capacity to 30 h/week for 1.2a	22.5 × 0.85 = 19.1 plannable h/week → 56.75 ÷ 19.1 ≈ **3.0 weeks** of work, then store review → **~week 4**	+50% hours (cost); **still not 3 weeks** because store review cannot be compressed by effort	Possible if Meera funds it and the public sign-up depends on it
D. Cut evidence work to fit 3 weeks	Appears to fit	Silently removing gate evidence (security, rollback, watch)	**Rejected**
E. Delay the public sign-up page until 1.2a passes its gates	Protects against launching growth on a risky release	Marketing delay	Recommended alongside A

Phasing costs about 7 hours of repeated release overhead: the full release's 87 raw hours become 45 (1.2a) + 49 (1.2b: the remaining 42 plus 7 of repeated release work) = **94 raw hours** across both releases. That is the price of delivering value earlier and containing risk.

Message to Meera:

> *"Thanks. Here is what I can do. The password-change part (R-1) is the one that matters for security, and I estimate about 5 weeks of work at 20 hours a week, then app store review, so it would be live around week 6. If you can fund 30 hours a week for this release, that comes down to roughly week 4; I can't make it three weeks, because even with extra hours the store review takes up to a week and I won't skip the security and rollback checks on a finance app. The session list and the 10-session limit (R-2, R-3) involve a database migration on 1.5 million rows that I want to rehearse carefully; I'd plan that as a second release a few weeks later. I'd also suggest holding the public sign-up page until 1.2a is live and checked. What I need from you: confirmation of this split, a one-hour slot in the final week for a*

> *runbook dry run, and a second phone for testing. I'll update the forecast after the first week with real numbers."*

7. Milestones and plan (recommended: 1.2a at 20 h/week)

Weekly plan (plannable 12.75 calibrated hours per week; hours shown are calibrated):

Week	Focus	Hours
1	B1 (9.9) + start B5-lite (2.6); book device and dry run; confirm sign-out behaviour	12.5
2	Finish B5-lite (4.0); start M1 (8.5 of 11.6)	12.5
3	Finish M1 (3.1); M3 (7.25); start M4 (2.15 of 5.8)	12.5
4	Finish M4 (3.65); Q1 (3.6); Q3 (3.6); start R1 (1.65 of 4.8)	12.5
5	Finish R1 (3.15), **submit for review;** R2 and verification (3.6)	6.75
6	Store review (elapsed); go-live; 48-hour watch	n/a
	Total	**56.75**

Milestones:

Milestone	End of	Exit criteria	Evidence
M1 Backend ready	Week 2	Password-change contract tests pass including concurrency and authorization; old tokens rejected after change	CI report
M2 App feature complete	Week 3	Change-password flow works on two real devices; other device signed out with the right message	Demo on two phones
M3 Gate G1 passed	Week 4	G1 evidence tied to the release commit (tests, security checks, performance sanity)	Evidence pack
M4 Gate G2 passed + submitted	Week 5	Rollback drill ⩽ 10 min using the runbook; dry run by Meera; store submission done; residual risks accepted	Drill log, dry-run note
M5 Live + G3	Week 6	48-hour watch clean	Dashboards

8. New risks created by the plan

ID	Risk (cause → event → impact)	L	I	Score	Owner	Early warning
P-1	Because mobile estimates have run 45% over, mobile work may again overrun → the date slips	4	3	12	Lead	Week-2 mobile hours vs plan
P-2	Because store review time varies from 1 to 7 days and a rejection restarts it → go-live slips	3	3	9	Meera	Review status by day 3
P-3	Because Meera plans the public sign-up page for right after the release → growth lands on an unproven release	3	4	12	Meera	Marketing dates vs gate status
P-4	Because the production-size copy and a second device are not yet arranged → B4 rehearsal and M4 testing stall	3	2	6	Lead	Not arranged by end of week 1
P-5	Because the lead works 20 hours a week with no backup → an absence stalls the release	3	4	12	Lead (flag to Meera)	Planned absences

What to notice in the reference. The honest answer was *not* "three weeks" and *not* "no." It was a phased plan with maths behind it, options for the client, the earliest realistic date including the part you cannot speed up (store review), and the safety evidence kept in scope. The same estimate that exposed the problem also supplied the solution.

Part 9 — Recall test, with answers

Cover the answers and try first.

Q1. State the 100% rule for a WBS and why it matters. *Answer:* The WBS must include all the work in scope and nothing outside it. Work not in the WBS is not estimated, scheduled, or noticed until it causes a delay.

Q2. Why derive the WBS from the risk register and the release gates, not only from the PRD? *Answer:* The mitigations (restore drills, authorization tests, probes) and the gate evidence (load report, drill logs) are real work. Omitting them gives a feature-only plan that is optimistic by roughly a factor of two, and the gates will either fail or be skipped.

Q3. Compute the PERT estimate and spread for O = 4, M = 8, P = 18. *Answer:* $E = (4 + 4\times8 + 18) \div 6 = 54 \div 6 =$ **9.0.** $\sigma = (18 - 4) \div 6 \approx$ **2.3.**

Q4. Why is summing PERT variances over a whole project misleading? *Answer:* It assumes independent errors that cancel. Real errors are correlated (same estimator, same unfamiliar

technology, forgotten scope), so they accumulate. The arithmetic spread is a lower bound; the real range is wider and should be set by judgment and replaced with measured data.

Q5. Distinguish effort, capacity, and duration. A 40-hour estimate, 20 hours available per week, focus factor 0.75: how long? *Answer:* Effort is hours of work; capacity is productive hours per week; duration is elapsed time. Capacity = 20 × 0.75 = 15 hours per week; duration = 40 ÷ 15 ≈ **2.7 weeks.**

Q6. With one engineer, what is the critical path, and what is the only real parallelism? *Answer:* The critical path is essentially all the effort in sequence (resource dependency). The only real parallelism is waiting items (store accounts, client decisions, store review, provisioning) that need calendar time but not your effort.

Q7. What is a walking skeleton and why build it first? *Answer:* The thinnest end-to-end implementation touching every component and deployed through the real pipeline. It proves the architecture and plumbing early, when surprises are cheap.

Q8. What is a spike and when do you use one? *Answer:* A time-boxed experiment whose output is knowledge. Use it when you cannot give a bounded range (the pessimistic estimate is several times the most likely), instead of guessing.

Q9. Remaining raw estimate is 90 hours, your accuracy ratio is 1.25, and you can plan 25 actual hours per week. How long? *Answer:* 90 × 1.25 = 112.5 actual hours; 112.5 ÷ 25 = **4.5 weeks.**

Q10. What distinguishes a good milestone from a vague one? *Answer:* It is demonstrable, has objective exit criteria and evidence, retires a specific risk, and (for late milestones) ties to a release gate. "70% done" is not a milestone.

Q11 (challenge). The client says: "8 weeks is a hard date." Your plan says 16. What do you do? *Answer:* Don't change the estimate to please anyone. Show the breakdown (including the non-feature work), the capacity maths, and what 8 weeks can hold. Offer options: phase the delivery (alpha at 8 weeks), move the date, raise capacity (with its limits and cost), or cut scope that does not touch the safety evidence, and refuse to cut evidence silently. Recommend one, list the client's dated tasks, and agree to re-forecast each sprint with real data.

Part 10 — Completion checklist

Tick honestly. Revisit the section noted for any you cannot.

- ☐ I can explain that a plan is a set of tools (expectation, feasibility, sequencing, baseline) and distinguish **estimate, forecast, commitment, and target** (3.1).
- ☐ I can build a **deliverable-based WBS** that satisfies the 100% rule and includes the forgotten categories (3.2).

- [ ] I can derive WBS packages from the **PRD, design, risk register, and gates** (3.2).
- [ ] I can create an ordered **backlog** with item types and explain **risk-first sequencing** and the **walking skeleton** (3.3).
- [ ] I can map **dependencies,** separate **effort from lead time,** and explain the critical path for a one-person team (3.4).
- [ ] I can give an estimate as a **range with assumptions,** and compute a **PERT** estimate (3.5).
- [ ] I can explain **why the summed spread is misleading** and how to set a realistic range (3.5).
- [ ] I can compute **capacity with a focus factor** and convert effort to duration (3.5).
- [ ] I can use **historical ratios** to calibrate remaining work (3.8, 8B).
- [ ] I can define **milestones** with exit criteria and tie them to **gates** (3.6).
- [ ] I can plan a **sprint:** goal, capacity at about 85%, ready items, tasks, and a done definition (3.7).
- [ ] I can **track and replan** with a burn-up, accuracy ratio, and re-forecast (3.8, 4.11).
- [ ] I can present **options with trade-offs** and deliver bad news early, specifically, and with a recommendation (4.7, 8B).
- [ ] I know which practices are **context-dependent** (story points vs hours, Scrum vs Kanban, sprint length) and why (6).

Closing: what you should understand before moving on

Before the next lecture, make sure you can say these five things in your own words:

1. **A plan is a set of tools, not a prediction.** It exposes feasibility early, sequences risk first, and gives you a baseline to track against.
2. **The WBS must contain all the work,** including foundations, quality evidence, recovery drills, and release work. A feature-only estimate flatters the date by almost a factor of two.
3. **An estimate is a range with assumptions,** and the arithmetic spread understates real uncertainty because errors are correlated. Use history to calibrate, and spikes to buy information.
4. **Effort, capacity, and elapsed time are different.** With one engineer, only waiting items run in parallel, so start the long-lead and client items on day one.
5. **Plans change by data, not by hope.** Re-forecast every sprint, tell the client early with options, and never quietly trade away the safety work.

This is the end of Module 4, Lecture 1. I'm stopping here, as instructed in the format of this course.